



HELM SCHEMATIC.

A blueprint-style reference for charts, templates, and releases — how a values file and a template directory become live Kubernetes manifests, drawn out.

SHEETS 01–13 SCOPE CHARTS + TEMPLATING + RELEASES REV 2026.1

HLM-01

What Helm Is

Helm is the package manager for Kubernetes. A **chart** bundles a set of Kubernetes manifests as parameterized templates, so one chart can be installed many times with different configuration — instead of hand-editing YAML for every environment.

- ▶ **Chart** — the package: templates + default configuration + metadata, distributable as a versioned artifact.
- ▶ **Release** — one installed instance of a chart in a cluster, with its own name and revision history.
- ▶ **Values** — the configuration layer that fills in a chart's templates; the same chart with different values produces different manifests.
- ▶ **Repository** — an index of packaged charts you can search, add, and install from, like a package registry for Kubernetes apps.

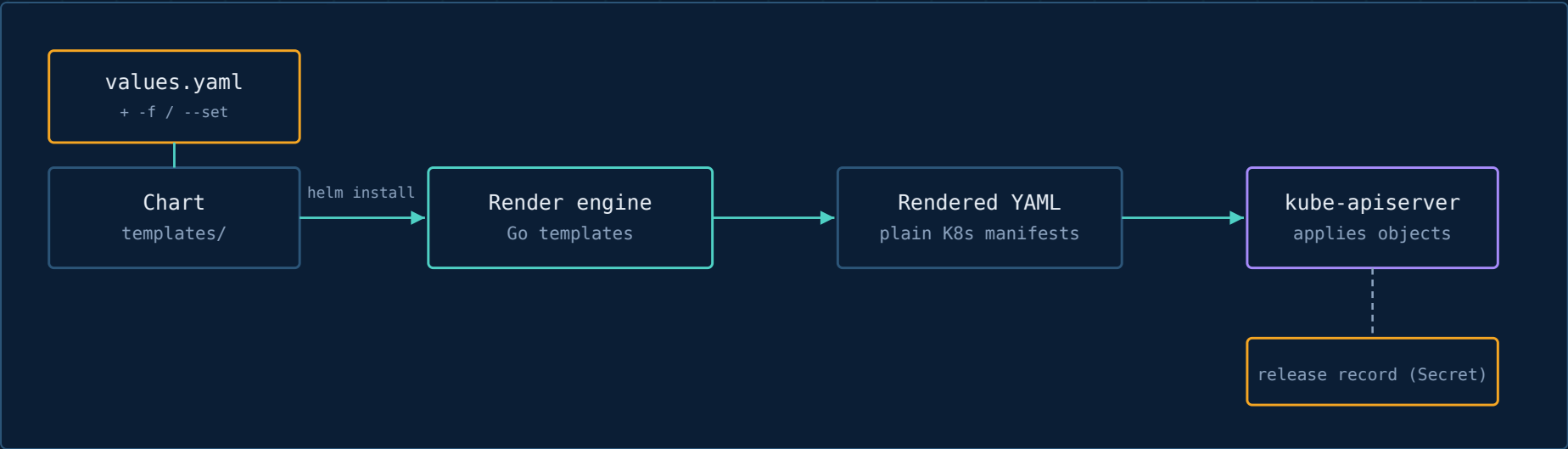
HELM 3, NOT HELM 2

Helm 3 removed Tiller, the old in-cluster server component. There's no cluster-side daemon to manage anymore — the Helm CLI talks to the Kubernetes API directly, using your existing kubeconfig and RBAC permissions.

HLM-02

Architecture: Client-Only

Helm is just a CLI. It renders templates locally, then submits the resulting manifests to the Kubernetes API like `kubectl apply` would — and stores a record of what it did as a Secret inside the cluster.



WHERE RELEASE STATE LIVES

Every install/upgrade stores a compressed record of that revision's manifests and values as a Kubernetes Secret, in the release's namespace — that's how `helm rollback` can reconstruct any prior revision without needing Helm's own database.

HLM-03

Chart Anatomy

A chart is a directory with a fixed, predictable shape. Helm looks for these exact names.

```
mychart/
├─ Chart.yaml # name, version, dependencies
├─ values.yaml # default configuration
├─ charts/ # subcharts / dependencies live here
├─ templates/ # Go-templated K8s manifests
│ └─ deployment.yaml
│ └─ service.yaml
│ └─ ingress.yaml
│ └─ _helpers.tpl # named template snippets, no output
│ └─ NOTES.txt # printed after install/upgrade
│ └─ tests/
│ └─ test-connection.yaml
└─ .helmignore # files to exclude when packaging
```

Chart.yaml	Required. Chart metadata — name, version, appVersion, dependencies.
values.yaml	Default values available to every template as <code>.Values</code> .
templates/	Every <code>.yaml</code> here is rendered through the Go template engine.
charts/	Vendored subcharts — either pulled via dependencies or placed manually.
_helpers.tpl	Defines reusable named templates (<code>{{ define }}</code>); produces no manifest itself.
crds/	CustomResourceDefinitions, installed before templates, never templated or upgraded automatically.

HLM-04

Chart.yaml & values.yaml

CHART.YAML

```
apiVersion: v2
name: mychart
description: A web app chart
type: application
version: 0.3.1      # chart version
appVersion: "2.1.0" # app version
dependencies:
- name: redis
  version: "18.x.x"
  repository: "https://charts.bitnami.com/bitnami"
```

VALUES.YAML

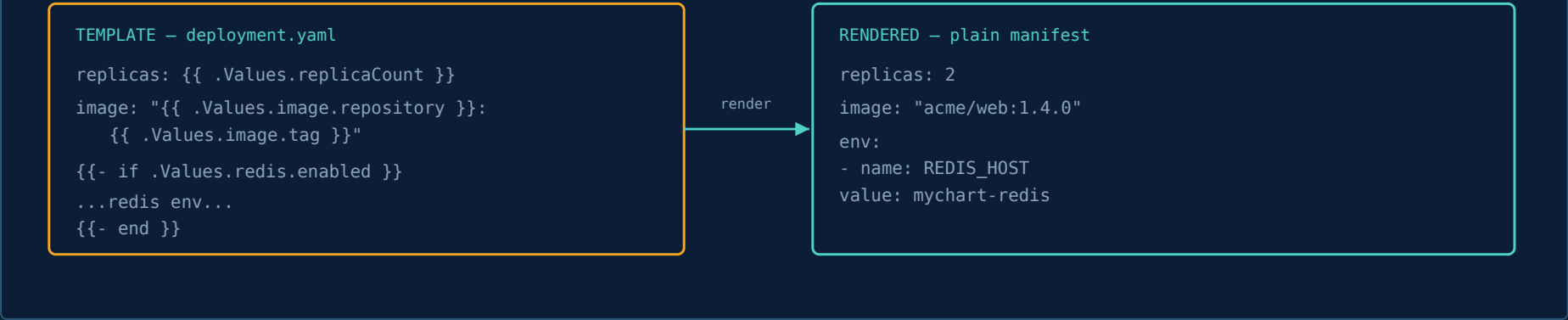
```
replicaCount: 2
image:
  repository: acme/web
  tag: "1.4.0"
service:
  type: ClusterIP
  port: 80
resources:
  requests:
    cpu: 100m
    memory: 128Mi
redis:
  enabled: true # passed into subchart
```

version is the chart's own release number (semver, required by Helm). **appVersion** is just an informational label for the version of the application it deploys — Helm never parses or enforces it.

HLM-05

Templates & Go Templating

Templates are plain Kubernetes YAML with `{{ }}` actions spliced in — Helm's Go template engine evaluates them against a data object built from values, chart metadata, and release info.

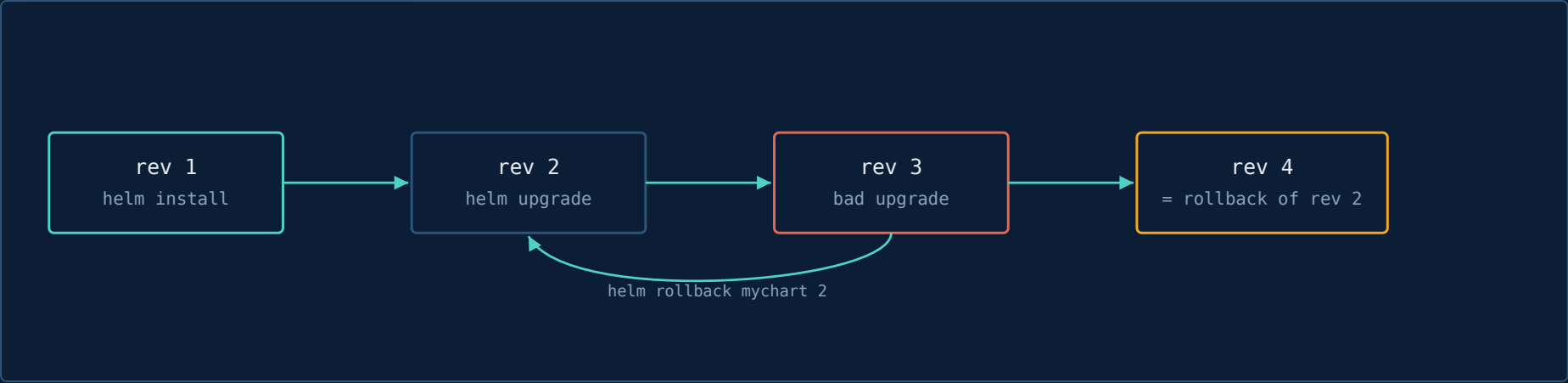


OBJECT	ACCESS
.Values	Merged values (chart defaults + overrides).
.Release	.Name, .Namespace, .Revision, .IsUpgrade, .IsInstall
.Chart	Contents of Chart.yaml — .Chart.Name, .Chart.Version
.Files	Non-template files in the chart, read with .Files.Get
.Capabilities	Kubernetes cluster version and supported API groups

- {{ if / else / end }}
- {{ range }}
- {{ with }}
- | default "x"
- | quote
- | nindent 4
- {{- -}} (whitespace trim)

HLM-06

Release Lifecycle



COMMAND	EFFECT
helm install NAME chart/	Renders + applies templates, creates revision 1.
helm upgrade NAME chart/	Renders with new values/chart, diffs against cluster, creates the next revision.
helm rollback NAME REV	Re-applies a prior revision's manifests as a brand-new revision — never rewinds history, only adds to it.
helm uninstall NAME	Deletes all release resources (unless <code>--keep-history</code> is set).

helm history NAME	Lists every revision, its status, and a one-line description.
helm status NAME	Shows the current state and re-prints NOTES.txt.

HLM-07

Values Override Precedence

Multiple sources can set the same value. Helm merges them, with later sources winning.



CLI

```
helm upgrade myapp ./mychart \  
  -f values-prod.yaml \  
  --set image.tag=1.4.1 \  
  --set replicaCount=5
```

MERGE, NOT REPLACE

Values merge key-by-key (deep merge for maps). A -f file only needs to specify the keys it's changing — everything else falls back to the chart's defaults.

HLM-08

Helpers & Named Templates

`_helpers.tpl` holds reusable snippets — most commonly consistent naming and label blocks — so every template stays DRY.

```
_HELPERS.TPL  
  
{{/* Fully qualified app name, chart+release, truncated to 63 chars */}}  
{{- define "mychart.fullname" -}}  
{{ printf "%s-%s" .Release.Name .Chart.Name | trunc 63 | trimSuffix "-" }}  
{{- end -}}  
  
{{- define "mychart.labels" -}}
```

```
app.kubernetes.io/name: {{ .Chart.Name }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}
```

```
TEMPLATES/DEPLOYMENT.YAML (USAGE)

metadata:
  name: {{ include "mychart.fullname" . }}
  labels:
    {{- include "mychart.labels" . | nindent 4 }}
```

DEFINE VS INCLUDE

`{{ define }}` declares a named template but renders nothing by itself. `{{ include "name" . }}` calls it and captures the output as a string — which is why it can be piped to filters like `nindent`, unlike the older `template` action.

Dependencies & Subcharts

PARENT CHART – mychart

values.yaml: redis.enabled: true → scoped to subchart "redis"
global.imageRegistry: myrepo.io → visible to ALL subcharts

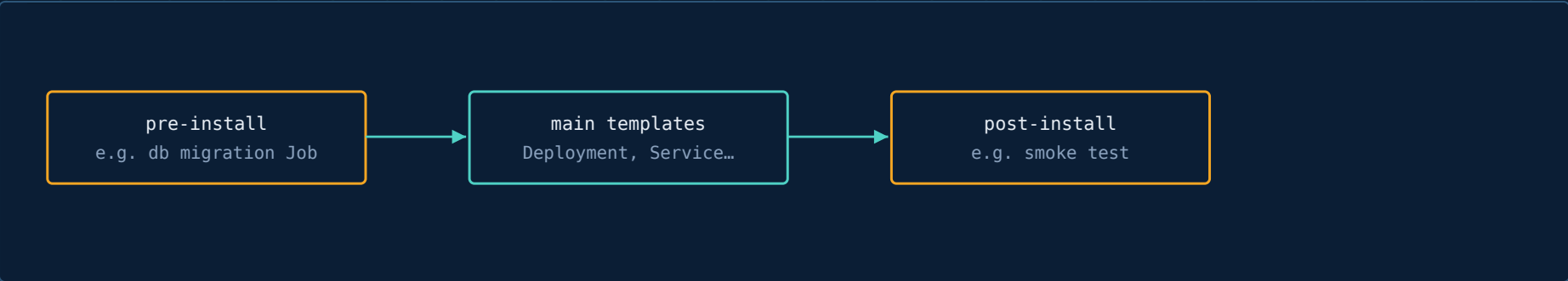


```
OPS

# declare in Chart.yaml dependencies:, then
helm dependency update ./mychart    # pulls into charts/, writes Chart.lock
helm dependency build  ./mychart    # rebuilds charts/ from Chart.lock
```

- ▶ **Scoping.** A subchart only sees the values nested under its own name in the parent's values.yaml, plus anything under the special `global` key.
- ▶ **condition** in Chart.yaml's dependency entry (e.g. `redis.enabled`) lets a parent turn a subchart on/off from values.yaml.
- ▶ Chart.lock pins exact dependency versions, the same role as a lockfile in other package managers.

A hook is a manifest annotated to run at a specific point in the release lifecycle — outside the normal install/upgrade ordering — commonly used for migrations, backups, or smoke tests.



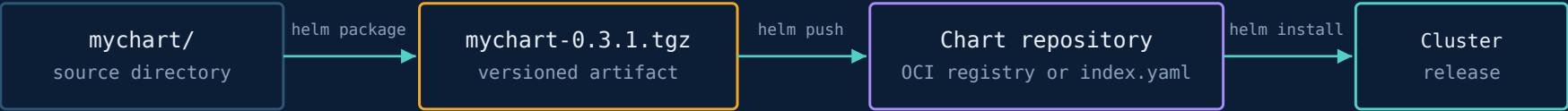
```
TEMPLATES/PRE-INSTALL-JOB.YAML

apiVersion: batch/v1
kind: Job
metadata:
  name: {{ .Release.Name }}-migrate
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "0"
    "helm.sh/hook-delete-policy": before-hook-creation
```

HOOK	FIRES
pre-install / post-install	Before / after all other manifests are created on install
pre-upgrade / post-upgrade	Before / after resources are updated on upgrade
pre-rollback / post-rollback	Around a rollback operation
pre-delete / post-delete	Around helm uninstall

NOT TRACKED AS RELEASE RESOURCES

Hook resources aren't part of the normal release object and aren't deleted automatically — set a hook-delete-policy or they'll accumulate across runs.



```
REPO & PACKAGE

# classic HTTP repo
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/redis

# OCI registry (Helm 3.8+)
helm push mychart-0.3.1.tgz oci://ghcr.io/acme/charts
helm install myapp oci://ghcr.io/acme/charts/mychart --version 0.3.1

# package a local chart
helm package ./mychart
```

HLM-12

Testing & Linting

COMMAND	WHAT IT CHECKS
helm lint ./mychart	Validates chart structure and template syntax; catches common mistakes before you ever touch a cluster.
helm template ./mychart	Renders manifests locally without installing anything — the fastest way to see exactly what would be applied.
helm install --dry-run --debug	Renders against the live cluster context (so <code>.Capabilities</code> resolves) without creating any resources.
helm test RELEASE	Runs manifests annotated <code>helm.sh/hook: test</code> — typically a Pod that curls the service and asserts a 200.

```
TEMPLATES/TESTS/TEST-CONNECTION.YAML

apiVersion: v1
kind: Pod
metadata:
  name: {{ .Release.Name }}-test-connection
  annotations:
    "helm.sh/hook": test
spec:
  containers:
    - name: wget
```



```
image: busybox
command: ['wget']
args: ['{{ include "mychart.fullname" . }}:{{ .Values.service.port }}']
restartPolicy: Never
```

Helm CLI Cheat Sheet

CREATE & INSPECT

```
# scaffold / inspect
helm create mychart
helm show values ./mychart
helm show chart ./mychart
helm template myapp ./mychart

# repos
helm repo add name url
helm repo list
helm search repo keyword
```

RELEASE OPS

```
# releases
helm install myapp ./mychart -f values.yaml
helm upgrade myapp ./mychart --set image.tag=2.0
helm upgrade --install myapp ./mychart # idempotent
helm rollback myapp 2
helm uninstall myapp

# state
helm list -A
helm history myapp
helm get values myapp
helm get manifest myapp
```

--dry-run --debug

--atomic

--wait

--create-namespace

--version 1.2.3